

UNIVERSITY OF COLOMBO SCHOOL OF COMPUTING

SCS 3311 — Mobile Application Design and Development

Mini-Project

HomeSense

Smart Home Monitoring and Control System

Defence Notes

Group Members

Name	Index number	Module
R.L. Weerasinghe	23002204	Synchronisation, simulator and safety worker
W.T. Mahagamage	23001038	Device profiles, control interface and reporting
D.M. Isakya	23000643	Floor representation and grid mapping

August 2026

How to use this document

Assessment includes individual defence: each member must answer questions on their own module. This document covers the five questions most likely to be asked of each member, with an answer for each, followed by questions that span modules and may be put to anyone.

Read your own section aloud until the answers can be given without reference to the page.

1 Questions any member may be asked

Why three fields rather than one boolean?

A single boolean cannot distinguish between a state that has been requested, a state that is actually in effect, and the system's confidence in that information. `desiredState` records the application's intent, `reportedState` records the hardware's confirmation, and `status` is the server's reconciliation of the two. This is what allows the error and disconnected states to represent observations rather than assumptions.

Where does the safety cut-off run, and why there?

In the worker process, on the server side. Implemented in the application it would only protect the house while the application was running, which defeats the purpose of a safety feature.

2 R.L. Weerasinghe (23002204) — synchronisation, simulator and safety

Owns `data/remote/`, `data/repository/`, `simulator/`, `worker/` and `database.rules.json`.

1. Why Realtime Database rather than Firestore?

Two reasons. Realtime Database provides per-child listeners, so toggling one slot produces a single callback carrying that device rather than a retransmission of the whole devices subtree, which matters for a grid of independently toggled slots. It also provides `onDisconnect()`, which lets the server record a client's disappearance, so the disconnected state is an observation rather than an inference made independently by each observer.

Firestore's document-level granularity and higher write latency suit a different access pattern. The cost is the absence of query support and a denormalised tree, which is acceptable because the tree is small, fixed and documented.

2. How is the absence of manual refresh actually guaranteed?

There is no refresh method to call. `HomeRepository` exposes no `refresh()`, `reload()` or `fetch()` operation. The device list is a `StateFlow` supplied by database child listeners, so a change from the simulator, the worker or another client arrives as a callback and triggers recomposition.

The repository tests demonstrate this by emitting a change on the source and asserting the flow carries it, without calling into the repository at all. It is a property of the design rather than a feature added to it.

3. Can a client actually not misreport a device's status?

No, and the mechanism is worth stating precisely. In Realtime Database a write grant propagates to all descendants of the node where it is declared and cannot be withdrawn lower down, so a nested `.write: false` would have no effect. Validation rules, however, do restrict client writes, and the Admin SDK bypasses validation. The rules use that asymmetry: `status`, `link`, `onSince` and `mismatchSince` carry validators that accept a write only when the value is unchanged. A client therefore cannot publish a status value, clear an error, or suppress a disconnected state, while the worker writes them freely.

The limitation is that the application and the simulator both authenticate anonymously, so the rules cannot distinguish them, and a modified client could write `reportedState` as though it were hardware. Separating the roles would require custom claims and a service to issue them. What no client can do is write `status`, which is what the safety argument depends on.

4. Why is the cut-off timer not implemented with `setTimeout`?

Because it would be lost when the process terminated, and lost without any indication. The `onSince` value is held in the database and elapsed time is recomputed from it at every evaluation, so a worker that crashes or is redeployed resumes correctly: the next pass observes an appliance forty-seven seconds into a thirty-second limit and switches it off immediately.

There is a test for that scenario and another confirming the cut-off fires at exactly `max_on_duration`. Two triggers drive the same pure function — child listeners for immediate reaction and a thirty-second sweep for recovery — so they cannot produce inconsistent results.

5. Why does the schedule rule act on boundaries rather than enforcing the interval?

Because enforcing the interval would override manual control. A user switching a scheduled light off at 19:00 would find it on again within a minute. Acting only at the on-minute and off-minute allows an override to persist until the next boundary, which matches what a timer is normally expected to do.

Application is idempotent, since a write occurs at a boundary minute only when the desired state differs from the target, so the sweep cannot apply a transition twice. The consequence is that a boundary occurring entirely while the worker is down is not applied retrospectively, which is stated in the report rather than left implicit.

3 W.T. Mahagamage (23001038) — device profiles and reporting

Owns `ui/device/`, `ui/report/`, `UsageCsv` and the Room aggregation queries.

1. Why is `max_on_duration` a property of a slot rather than of a device type?

Because an iron is connected to an ordinary socket and a scheduled lamp may be wired to one channel of a gang box. Modelling `HAZARD` and `LIGHT` as device kinds would make neither arrangement expressible without a separate type for every combination. A slot is a controllable point; the appliance is assigned to it, and both the appliance's properties and its policies belong at that point. The specification's phrasing, "specialised slots assigned to appliances", indicates the same structure.

2. The toggles are optimistic. Is that not misleading, given that only the server knows the actual state?

It would be if nothing followed. The switch changes position immediately because waiting for a round trip makes the interface feel unresponsive, but the application writes only `desiredState` and cannot operate a relay. The optimistic position is held for at most four seconds, and if `reportedState` has not followed, the control reverts and the user is informed.

The four seconds is chosen to be longer than a normal round trip and shorter than the worker's ten-second error threshold, so the user is warned shortly before the status indicator changes.

3. Why aggregate usage in SQL rather than over the database snapshot?

Two reasons. It works without a network connection, since Room is local, so the reporting screen is populated offline. And it scales: summing thousands of events in application code would require holding the entire log in memory and recomputing on every emission, whereas SQL performs the aggregation in the database against an index.

The query sums `durationSec` from off and cut-off events only, since those are the events that close an on-period, so no state reconstruction is required.

4. Why not derive usage by comparing successive state snapshots?

Because that would omit every transition occurring while the application was closed, which is exactly the period the safety worker covers. An appliance switched on at 14:02 and cut off at 14:32 with the phone unavailable must appear in the report, and it does, because the worker logged those events when they occurred.

Usage is append-only and each component logs its own transitions. Comparison would also double-count after a reconnection; upserting on the database push key makes replay idempotent.

5. Why is the energy figure described as an estimate?

Because it is one. It multiplies the appliance's rated wattage by its on-time, which assumes full rated draw throughout. A kettle does not behave that way: it draws heavily while heating and very little afterwards. Measuring properly would require a metering device reporting actual consumption, which is outside the scope of a simulation.

Labelling it an estimate wherever it appears is the accurate description, and accepting a measured value

instead would be a one-line change.

4 D.M. Isakya (23000643) — floor representation

Owns `ui/plan/`, `ui/floors/`, `GridMapper` and `FloorPlanSpec`.

1. Why store grid cells rather than pixel coordinates?

Pixel coordinates are not meaningful across devices: the same offset falls in a different room on a tablet than on a small phone, and rotating the screen would move every device. A cell is a stable identity — cell (4, 2) is the same location in the house regardless of what is rendering it.

Translation to pixels happens once, at draw time, from the current canvas size. It also means the database stores information about the house rather than about one particular screen.

2. Why is `GridMapper` a separate class rather than arithmetic inside the composable?

Because it is the part that can be wrong in a way that is not visible on inspection. Compose code requires a device or a preview to exercise, whereas a pure class with no Android dependency runs as a JVM unit test in milliseconds.

There are sixteen tests: the round trip for every cell, taps on the letterbox margins, taps exactly on a cell boundary and on the plan edge, a zero-sized canvas, and the property that a given cell resolves identically under phone and tablet geometry. Inline, none of that would be tested, and a placement error would first appear during the demonstration.

3. What is the letterboxing, and why does it matter?

The plan has a fixed aspect ratio and the available canvas generally does not, so the plan is fitted within the canvas with its ratio preserved, leaving margins on two sides. All cell arithmetic is then performed relative to the fitted rectangle rather than the raw canvas bounds.

Without that, the grid would stretch with the window and devices would drift out of their rooms as the layout changed. `cellAt()` also returns null for a tap on the margins, which represents “not on the plan” rather than “outside the grid”.

4. Why are plans drawn as geometry rather than supplied as images?

Rendering stays sharp at any screen density with no bitmap scaling; the aspect ratio is exact, which makes the fitting calculation exact; and because the plans are original work, there is no third-party image licence to attribute. `FloorPlanSpec` declares rooms, doorways and windows in unit-less coordinates, and Compose Canvas renders them.

5. How does the plan represent a gang box with a fault on one channel?

The marker shows the most severe of its slot statuses, not an average and not the first. A fault on one channel must not be concealed by two working channels, so a three-gang box with one error is drawn as an error marker.

Shape carries the device kind — circle, square with one indicator per slot, camera outline — and fill pattern carries the status: solid, outline, cross, dashed. Nothing depends on colour alone, which matters both for accessibility and for legibility when projected.

5 Further questions to prepare

What happens if two people toggle the same switch simultaneously?

The last write to `desiredState` takes effect, and both clients converge on whatever the hardware subsequently reports. No locking is required, because the relay's report rather than either client's intent is the source of truth for `status`.

Why does the application still show disconnected for a device whose relay is on?

Because that is not known. The heartbeat has stopped, so the last reported state is stale by an unknown interval, and reporting on would present an assumption as an observation. The `onSince` value is retained across a disconnection, so that if the node returns while still running, the cut-off counts from when it actually started rather than from the reconnection.

Room was not required by the specification. Why is it used?

For offline availability and for usage aggregation, and the report states this rather than implying it was required. Live device status is not cached, since an outdated on indicator is more misleading than an honest disconnected one.

What happens if the worker is not running?

No status is derived, no cut-off occurs and no schedule is applied. This is a genuine single point of failure and it is documented as one. The simulator displays the worker's own heartbeat and reports when it is not running.

Where does the application write status?

It does not, anywhere. `RemoteHomeSource`, which defines the complete set of operations the application can perform against the database, contains no method that writes it. The database rules enforce the same restriction independently.